

AI ASSISTED CODING

ASSIGNMENT-5.1 AND 6

NAME: Geethanjali

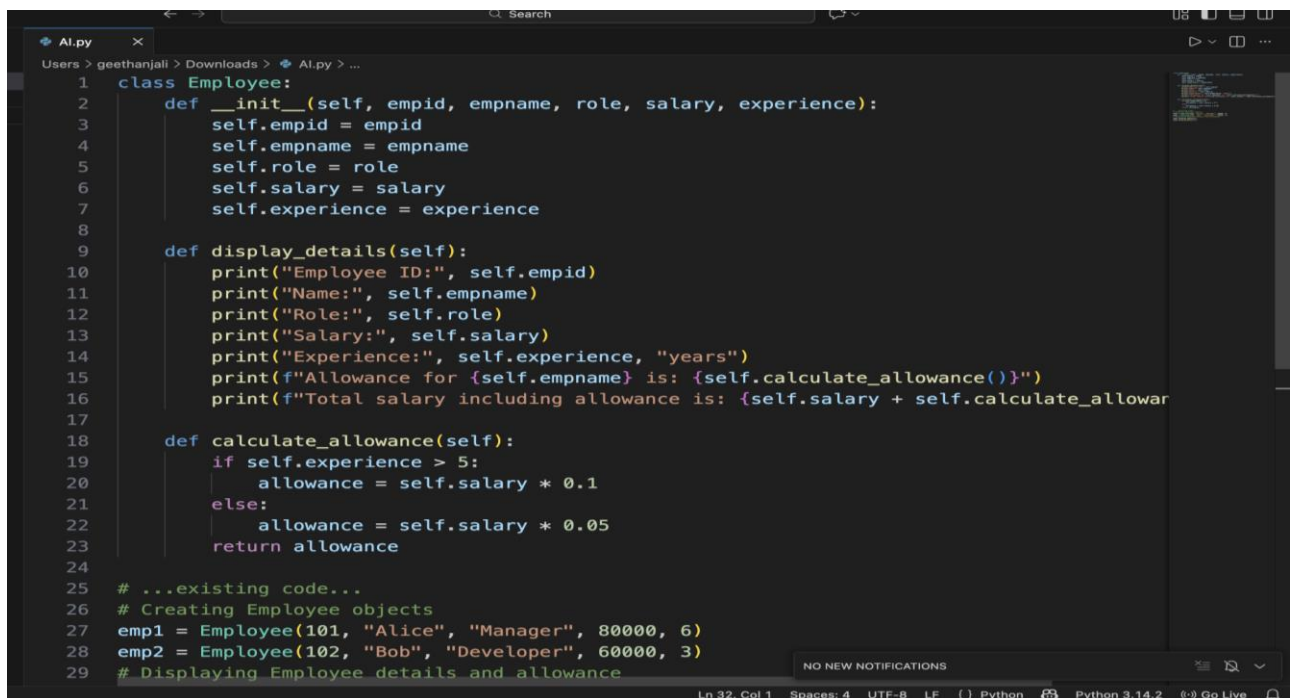
H.T.No.:2303A51005

Task 1:

Employee Data: Create Python code that defines a class named 'Employee' with the following attributes: 'empid', 'empname', 'designation', 'basic_salary', and 'exp'. Implement a method 'display_details()' to print all employee details. Implement another method 'calculate_allowance()' to determine additional allowance based on experience:

- If 'exp > 10 years' → allowance = 20% of 'basic_salary'
- If '5 ≤ exp ≤ 10 years' → allowance = 10% of 'basic_salary'
- If 'exp < 5 years' → allowance = 5% of 'basic_salary'

Finally, create at least one instance of the 'Employee' class, call the 'display_details()' method, and print the calculated allowance.



```
1 class Employee:
2     def __init__(self, empid, empname, role, salary, experience):
3         self.empid = empid
4         self.empname = empname
5         self.role = role
6         self.salary = salary
7         self.experience = experience
8
9     def display_details(self):
10        print("Employee ID:", self.empid)
11        print("Name:", self.empname)
12        print("Role:", self.role)
13        print("Salary:", self.salary)
14        print("Experience:", self.experience, "years")
15        print(f"Allowance for {self.empname} is: {self.calculate_allowance()}")
16        print(f"Total salary including allowance is: {self.salary + self.calculate_allowance()}")
17
18    def calculate_allowance(self):
19        if self.experience > 10:
20            allowance = self.salary * 0.2
21        elif 5 <= self.experience <= 10:
22            allowance = self.salary * 0.1
23        else:
24            allowance = self.salary * 0.05
25        return allowance
26
27 # ...existing code...
28 # Creating Employee objects
29 emp1 = Employee(101, "Alice", "Manager", 80000, 6)
30 emp2 = Employee(102, "Bob", "Developer", 60000, 3)
31 # Displaying Employee details and allowance
```

```
1 class Employee:
16     print(f"Total salary including allowance is: {self.salary + self.calculate_allowance}")
17
18     def calculate_allowance(self):
19         if self.experience > 5:
20             allowance = self.salary * 0.1
21         else:
22             allowance = self.salary * 0.05
23         return allowance
24
25 # ...existing code...
26 # Creating Employee objects
27 emp1 = Employee(101, "Alice", "Manager", 80000, 6)
28 emp2 = Employee(102, "Bob", "Developer", 60000, 3)
29 # Displaying Employee details and allowance
30 emp1.display_details()
31 emp2.display_details()
32
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/local/bin/python3 /Users/geethanjali/Downloads/AI.py
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI.py
Employee ID: 101
Name: Alice
Role: Manager
Salary: 80000
Experience: 6 years
Allowance for Alice is: 8000.0
Total salary including allowance is: 88000.0
Employee ID: 102
Name: Bob
Role: Developer
Salary: 60000
Experience: 3 years
Allowance for Bob is: 3000.0
Total salary including allowance is: 63000.0
geethanjali@Geethanjali / %
```

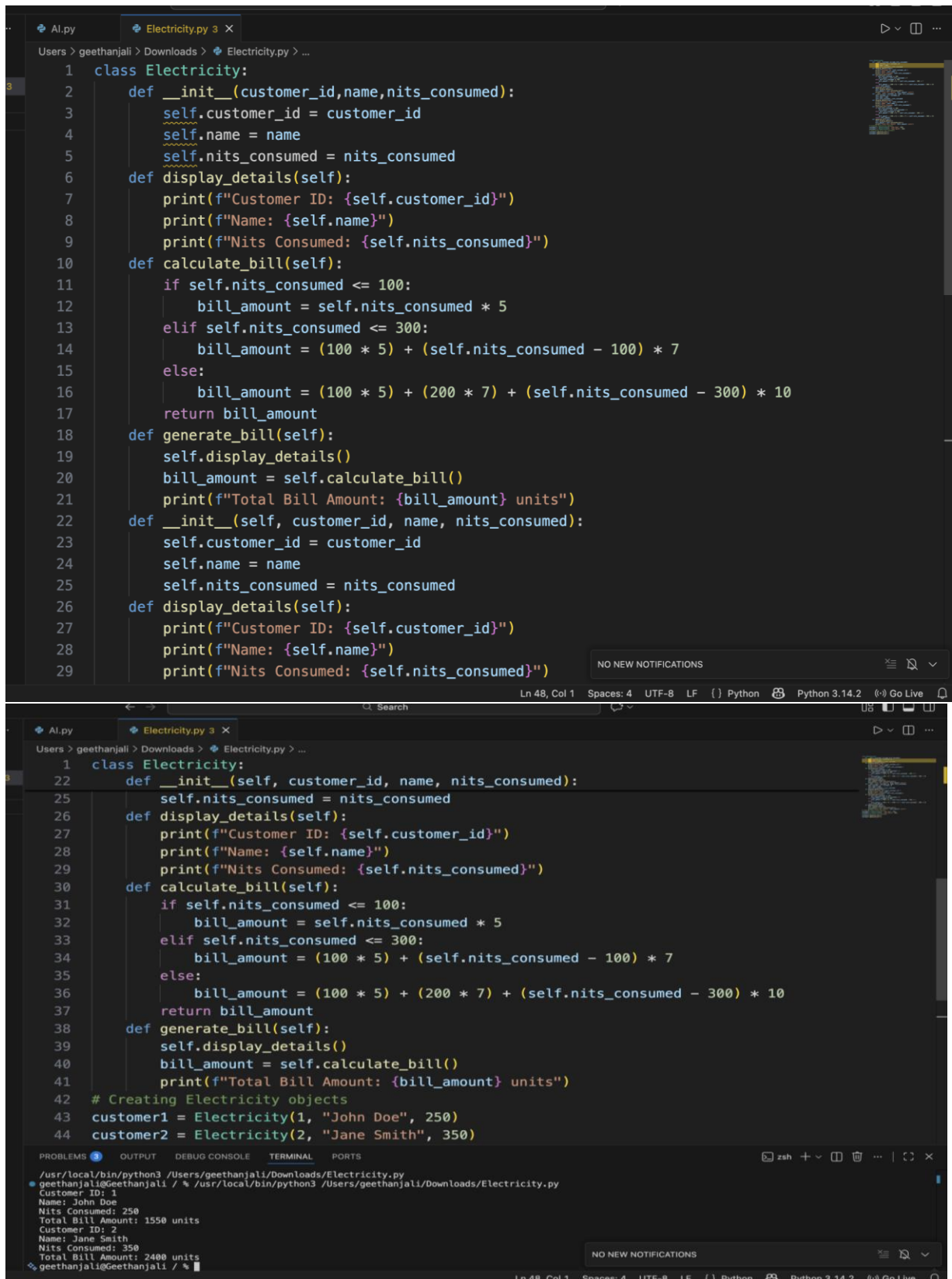
Ln 32, Col 1 Spaces: 4 UTF-8 LF Python Python 3.14.2 Go Live

Task 2:

Electricity Bill Calculation- Create Python code that defines a class named `ElectricityBill` with attributes: `customer\_id`, `name`, and `units\_consumed`. Implement a method `display\_details()` to print customer details, and a method `calculate\_bill()` where:

- Units $\leq 100 \rightarrow$ ₹5 per unit
- 101 to 300 units $\rightarrow$ ₹7 per unit
- More than 300 units $\rightarrow$ ₹10 per unit

Create a bill object, display details, and print the total bill amount.



```
1 class Electricity:
2     def __init__(customer_id,name,nits_consumed):
3         self.customer_id = customer_id
4         self.name = name
5         self.nits_consumed = nits_consumed
6     def display_details(self):
7         print(f"Customer ID: {self.customer_id}")
8         print(f"Name: {self.name}")
9         print(f"Nits Consumed: {self.nits_consumed}")
10    def calculate_bill(self):
11        if self.nits_consumed <= 100:
12            bill_amount = self.nits_consumed * 5
13        elif self.nits_consumed <= 300:
14            bill_amount = (100 * 5) + (self.nits_consumed - 100) * 7
15        else:
16            bill_amount = (100 * 5) + (200 * 7) + (self.nits_consumed - 300) * 10
17        return bill_amount
18    def generate_bill(self):
19        self.display_details()
20        bill_amount = self.calculate_bill()
21        print(f"Total Bill Amount: {bill_amount} units")
22    def __init__(self, customer_id, name, nits_consumed):
23        self.customer_id = customer_id
24        self.name = name
25        self.nits_consumed = nits_consumed
26    def display_details(self):
27        print(f"Customer ID: {self.customer_id}")
28        print(f"Name: {self.name}")
29        print(f"Nits Consumed: {self.nits_consumed}")
```

```
22    def __init__(self, customer_id, name, nits_consumed):
25        self.nits_consumed = nits_consumed
26    def display_details(self):
27        print(f"Customer ID: {self.customer_id}")
28        print(f"Name: {self.name}")
29        print(f"Nits Consumed: {self.nits_consumed}")
30    def calculate_bill(self):
31        if self.nits_consumed <= 100:
32            bill_amount = self.nits_consumed * 5
33        elif self.nits_consumed <= 300:
34            bill_amount = (100 * 5) + (self.nits_consumed - 100) * 7
35        else:
36            bill_amount = (100 * 5) + (200 * 7) + (self.nits_consumed - 300) * 10
37        return bill_amount
38    def generate_bill(self):
39        self.display_details()
40        bill_amount = self.calculate_bill()
41        print(f"Total Bill Amount: {bill_amount} units")
42    # Creating Electricity objects
43    customer1 = Electricity(1, "John Doe", 250)
44    customer2 = Electricity(2, "Jane Smith", 350)
```

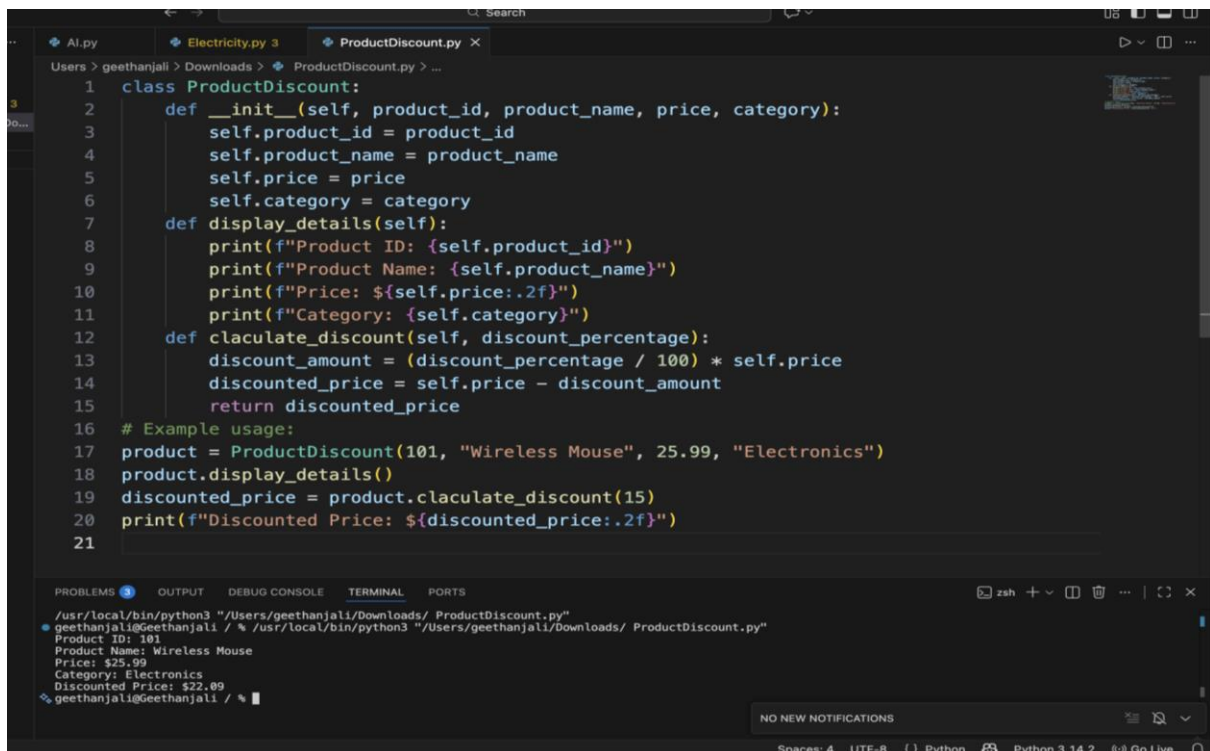
```
zsh
/usr/local/bin/python3 /Users/geethanjali/Downloads/Electricity.py
geethanjali@Geethanjali: / % /usr/local/bin/python3 /Users/geethanjali/Downloads/Electricity.py
Customer ID: 1
Name: John Doe
Nits Consumed: 250
Total Bill Amount: 1550 units
Customer ID: 2
Name: Jane Smith
Nits Consumed: 350
Total Bill Amount: 2400 units
geethanjali@Geethanjali: / %
```

Task 3:

Product Discount Calculation- Create Python code that defines a class named 'Product' with attributes: 'product_id', 'product_name', 'price', and 'category'. Implement a method 'display_details()' to print product details. Implement another method 'calculate_discount()' where:

- Electronics → 10% discount
- Clothing → 15% discount
- Grocery → 5% discount

Create at least one product object, display details, and print the final price after discount.



```
1 class ProductDiscount:
2     def __init__(self, product_id, product_name, price, category):
3         self.product_id = product_id
4         self.product_name = product_name
5         self.price = price
6         self.category = category
7     def display_details(self):
8         print(f"Product ID: {self.product_id}")
9         print(f"Product Name: {self.product_name}")
10        print(f"Price: ${self.price:.2f}")
11        print(f"Category: {self.category}")
12    def calculate_discount(self, discount_percentage):
13        discount_amount = (discount_percentage / 100) * self.price
14        discounted_price = self.price - discount_amount
15        return discounted_price
16
17 # Example usage:
18 product = ProductDiscount(101, "Wireless Mouse", 25.99, "Electronics")
19 product.display_details()
20 discounted_price = product.calculate_discount(15)
21 print(f"Discounted Price: ${discounted_price:.2f}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/local/bin/python3 "/Users/geethanjali/Downloads/ ProductDiscount.py"
geethanjali@geethanjali / % /usr/local/bin/python3 "/Users/geethanjali/Downloads/ ProductDiscount.py"
Product ID: 101
Product Name: Wireless Mouse
Price: $25.99
Category: Electronics
Discounted Price: $22.09
geethanjali@geethanjali / %
```

NO NEW NOTIFICATIONS

Spaces: 4 UTF-8 Python Python 3.14.2 Go Live

Task 4:

Book Late Fee Calculation- Create Python code that defines a class named 'LibraryBook' with attributes: 'book_id', 'title', 'author', 'borrower', and 'days_late'. Implement a method 'display_details()' to print book details, and a method 'calculate_late_fee()' where:

- Days late ≤ 5 → ₹5 per day
- 6 to 10 days late → ₹7 per day
- More than 10 days late → ₹10 per day

Create a book object, display details, and print the late fee.

```
1 class LibraryBook:
2     def __init__(self, book_id, title, author, borrower=None, days_late=0):
3         self.book_id = book_id
4         self.title = title
5         self.author = author
6         self.borrower = borrower
7         self.days_late = days_late
8     def info(self):
9         print(f"Book ID: {self.book_id}")
10        print(f"Title: {self.title}")
11        print(f"Author: {self.author}")
12        if self.borrower:
13            print(f"Borrower: {self.borrower}")
14            print(f"Days Late: {self.days_late}")
15        else:
16            print("This book is currently available.")
17    def calculate_fine(self):
18        fine_per_day = 0.50 # Assuming a fine of $0.50 per day late
19        total_fine = self.days_late * fine_per_day
20        return total_fine
21    def generate_report(self):
22        self.info()
23        if self.borrower and self.days_late > 0:
24            fine = self.calculate_fine()
25            print(f"Total Fine: ${fine:.2f}")
26        elif self.borrower:
27            print("No fine. Book returned on time.")
28    # Example usage:
29    book1 = LibraryBook(1, "1984", "George Orwell", "Alice", 3)
```

```
17 def calculate_fine(self):
18     fine_per_day = 0.50 # Assuming a fine of $0.50 per day late
19     total_fine = self.days_late * fine_per_day
20     return total_fine
21 def generate_report(self):
22     self.info()
23     if self.borrower and self.days_late > 0:
24         fine = self.calculate_fine()
25         print(f"Total Fine: ${fine:.2f}")
26     elif self.borrower:
27         print("No fine. Book returned on time.")
28 # Example usage:
29 book1 = LibraryBook(1, "1984", "George Orwell", "Alice", 3)
30 book2 = LibraryBook(2, "To Kill a Mockingbird", "Harper Lee")
31 book1.generate_report()
32 book2.generate_report()
33
```

geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/LibraryBook.py

Title: 1984
Author: George Orwell
Borrower: Alice
Days Late: 3
Total Fine: \$1.50
Book ID: 2
Title: To Kill a Mockingbird
Author: Harper Lee
This book is currently available.

Task 5:

Student Performance Report - Define a function `student\_report(student\_data)` that accepts a dictionary containing student names and their marks. The function should:

- Calculate the average score for each student
- Determine pass/fail status (pass ≥ 40)
- Return a summary report as a list of dictionaries

Use Copilot suggestions as you build the function and format the output.

```

106 def student_report(student_marks):
107     report=[]
108     for name,marks in student_marks.items():
109         avg_marks= sum(student_marks.values())/len(student_marks)
110         if marks>=40:
111             status="Pass"
112         else:
113             status="Fail"
114         report.append(("name": name, "marks": marks, "average": avg_marks, "status": status))
115     return report
116 marks={"Alice":85, "Bob":35, "charlie":55, "David":70, "Eva":90}
117 report=student_report(marks)
118 print(report)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python + - [] ... [] X

PS C:\Users\gudakh & C:\Python314\python.exe c:/Users/gudakh/OneDrive/Documents/AIAC/Lab_assignment_5_1.py

```

[('name': 'Alice', 'marks': 85, 'average': 67.0, 'status': 'Pass'), ('name': 'Bob', 'marks': 35, 'average': 67.0, 'status': 'Fail'), ('name': 'Charlie', 'marks': 55, 'average': 67.0, 'status': 'Pass'), ('name': 'David', 'marks': 70, 'average': 67.0, 'status': 'Pass'), ('name': 'Eva', 'marks': 90, 'average': 67.0, 'status': 'Pass')]

```

Task 6:

Taxi Fare Calculation-Create Python code that defines a class named 'TaxiRide' with attributes: 'ride_id', 'driver_name', 'distance_km', and 'waiting_time_min'. Implement a method 'display_details()' to print ride details, and a method 'calculate_fare()' where:

- ₹15 per km for the first 10 km
- ₹12 per km for the next 20 km
- ₹10 per km above 30 km
- Waiting charge: ₹2 per minute

Create a ride object, display details, and print the total fare.


```
121 class TaxiRide:
122     def __init__(self,ride_id,driver_name,distance_km,waiting_time_min):
123         self.ride_id = ride_id
124         self.driver_name = driver_name
125         self.distance_km = distance_km
126         self.waiting_time_min = waiting_time_min
127     def display_details(self):
128         print(f"Ride ID: {self.ride_id}")
129         print(f"Driver Name: {self.driver_name}")
130         print(f"Distance (km): {self.distance_km}")
131         print(f"Waiting Time (min): {self.waiting_time_min}")
132     def calculate_fare(self):
133         if self.distance_km <=10:
134             fare = self.distance_km * 15
135         elif 11 <= self.distance_km <=30:
136             fare = (10 * 15) + (self.distance_km - 10) * 12
137         else:
138             fare = (10 * 15) + (20 * 12) + (self.distance_km - 30) * 10
139         fare += self.waiting_time_min * 2
140         return fare
141 ride = TaxiRide(501, "Charlie Brown", 25, 10)
142 ride.display_details()
143 fare = ride.calculate_fare()
144 print(f"Total Fare: {fare}")
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_5_1.py
Ride ID: 501
Driver Name: Charlie Brown
Distance (km): 25
Waiting Time (min): 10
Total Fare: 350
```

Task 7:

Statistics Subject Performance - Create a Python function `statistics\_subject(scores\_list)` that accepts a list of 60 student scores and computes key performance statistics. The function should return the following:

- Highest score in the class
- Lowest score in the class
- Class average score
- Number of students passed (score ≥ 40)
- Number of students failed (score < 40)

Allow Copilot to assist with aggregations and logic

```
147 def statistics_subject(score_list):
148     total = sum(score_list)
149     average = total / len(score_list)
150     highest = max(score_list)
151     lowest = min(score_list)
152     passed = 0
153     failed=0
154     for i in score_list:
155         if i>=40:
156             passed+=1
157         else:
158             failed+=1
159     print(f"Number of Students Passed: {passed}")
160     print(f"Number of Students Failed: {failed}")
161     return { "average": average, "highest": highest, "lowest": lowest}
162 scores = [ 20, 40, 33, 72, 15, 60, 95, 40, 53, 81, 22, 47, 68, 79, 34, 91, 44, 58, 73, 38, 66, 84, 29, 50, 77, 92, 41, 36, 65, 80, 54, 87, 30, 69, 45, 71, 39, 83, 59, 74]
163 stats = statistics_subject(scores)
164 print(stats)
165
166
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_5_1.py
Number of Students Passed: 46
Number of Students Failed: 14
{'average': 58.6, 'highest': 95, 'lowest': 15}
```

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Task Description #8 (Transparency in Algorithm Optimization)

Task: Use AI to generate two solutions for checking prime numbers:

- Naive approach(basic)
- Optimized approach

Prompt:

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

Expected Output:

- Code for both methods.
- Transparent explanation of time complexity.
- Comparison highlighting efficiency improvements.

```
167 #generate two programs naive approach and optimized approach to check if given number is prime or not also calculate time and space complexities of both programs
168 import time
169 # Naive Approach
170 def is_prime_naive(n):
171     if n <= 1:
172         return False
173     for i in range(2, n):
174         if n % i == 0:
175             return False
176     return True
177 start_time = time.time()
178 number = 29
179 result_naive = is_prime_naive(number)
180 end_time = time.time()
181 print(f"Naive Approach: Is {number} prime? {result_naive}")
182 print(f"Time taken (Naive): {end_time - start_time} seconds")
183 # Time Complexity: O(n)
184 # Space Complexity: O(1)
185
186
187
188
189 # Optimized Approach
190 def is_prime_optimized(n):
191     if n <= 1:
192         return False
193     if n <= 3:
194         return True
195     if n % 2 == 0 or n % 3 == 0:
196         return False
197     i = 5
198     while i * i <= n:
199         if n % i == 0 or n % (i + 2) == 0:
200             return False
201         i += 6
202     return True
203 start_time = time.time()
204 result_optimized = is_prime_optimized(number)
205 end_time = time.time()
206 print(f"Optimized Approach: Is {number} prime? {result_optimized}")
207 print(f"Time taken (Optimized): {end_time - start_time} seconds")
208 # Time Complexity: O(√n)
209 # Space Complexity: O(1)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python +

PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/ATAC/Lab_assignment_5_1.py

Naive Approach: Is 29 prime? True
Time taken (Naive): 1.5497207641601562e-05 seconds
Optimized Approach: Is 29 prime? True
Time taken (Optimized): 1.0967254638671875e-05 seconds

Task Description #9 (Transparency in Recursive Algorithms)

Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.

Expected Output:

- Well-commented recursive code.
- Clear explanation of how recursion works.
- Verification that explanation matches actual execution.

```
213 #generate python code that develop fibonacci sequence using recursion
214 #Provide well commented explanation of each step
215 def fibonacci(n):
216     # Base case: if n is 0 or 1, return n
217     if n <= 1:
218         return n
219     else:
220         # Recursive case: return the sum of the two preceding numbers
221         return fibonacci(n - 1) + fibonacci(n - 2)
222 # Number of terms in the Fibonacci sequence
223 num_terms = 10
224 print(f"Fibonacci sequence up to {num_terms} terms:")
225 for i in range(num_terms):
226     print(fibonacci(i), end=" ")
227 # The above code defines a recursive function to generate Fibonacci numbers.
228
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_5_1.py
Fibonacci sequence up to 10 terms:
0 1 1 2 3 5 8 13 21 34
```

Task Description #10 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and processes data.

Prompt:

“Generate code with proper error handling and clear explanations for each exception.”

Expected Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.
- Validation that explanations align with runtime behavior.

```

229 #generate a program that reads a file and process the data
230 #Generate code with proper error handling and clear explanations for each exception.
231 def read_file(file_path):
232     try:
233         # Attempt to open the file
234         with open(file_path, 'r') as file:
235             data = file.read()
236             print("File content successfully read.")
237             return data
238     except FileNotFoundError:
239         # Handle the case where the file does not exist
240         print(f"Error: The file at {file_path} was not found.")
241     except PermissionError:
242         # Handle the case where there are permission issues
243         print(f"Error: You do not have permission to read the file at {file_path}.")
244     except Exception as e:
245         # Handle any other exceptions that may occur
246         print(f"An unexpected error occurred: {e}")
247 file_path = 'example.txt' # Specify the path to your file here
248 file_content = read_file(file_path)
249 if file_content:
250     print("File Content:")
251     print(file_content)

```

```

File content successfully read.
File Content:
Hello Everyone
Welcome to AI Assisted Coding class
Third year second semester
SR University
Lets work with files as part of lab assignment

```